CHAPTER 02

AGENTIC AI · LANGGRAPH SERIES

Agentic AI — Formal Study

A rigorous deep-dive into what Agentic AI really is — its formal definition, the six key characteristics that identify any agentic system, and the five core architectural components every LangGraph application is built upon.

WHAT THIS CHAPTER COVERS

- Formal definition of Agentic AI
- Reactive vs. proactive AI paradigms
- Autonomy & how to control it
- Goal-oriented operation & goal structure
- Planning: generate, evaluate, select
- Reasoning across planning & execution
- Adaptability to failures & feedback
- Context awareness & memory types
- Five components: Brain, Orchestrator, Tools, Memory, Supervisor

Agentic AI — Formal Study

RECAP — CHAPTER 01

Chapter 1 introduced the series by contrasting **Generative AI** (reactive, prompt-response chatbots) with **Agentic AI** (autonomous, goal-driven systems). A practical HR-recruiter scenario illustrated how a single high-level goal can drive a multi-step autonomous workflow. This chapter now formalises that intuition with precise definitions, characteristics, and architecture.

1. What Is Agentic AI?

Definition

Agentic AI is a software paradigm in which a system is given a high-level **goal** and then independently **plans**, **acts**, and **adapts** to achieve that goal — with minimal human instruction at each step. Formally:

> *"Agentic AI is a type of AI that can take up a task or goal from a user and then work towards completing it on its own with minimal human guidance. It plans, takes actions, adapts to changes, and seeks help only when necessary."*

In practical terms: you provide the *what*, and the system figures out the *how*. All planning, sub-task decomposition, tool selection, and execution happen autonomously inside the agent.

The Core Distinction: Reactive vs. Proactive

The sharpest way to understand Agentic AI is to contrast it with the **reactive** model of conventional Generative AI chatbots.

Aspect	Generative AI (Reactive)	Agentic AI (Proactive)
Input model	One prompt → one response	One goal → autonomous multi-step workflow
Initiative	Human must ask at every step	Agent drives steps on its own
Planning	None — answers the question as given	Creates and follows its own plan
Adaptation	None — responds to exactly what is asked	Detects problems and changes approach
Tool use	Optional add-on; human selects	Core capability; agent selects autonomously
Memory	Limited to current context window	Short-term and long-term memory across sessions
Human effort	High — every step requires a new prompt	Low — goal is set once; agent manages the rest

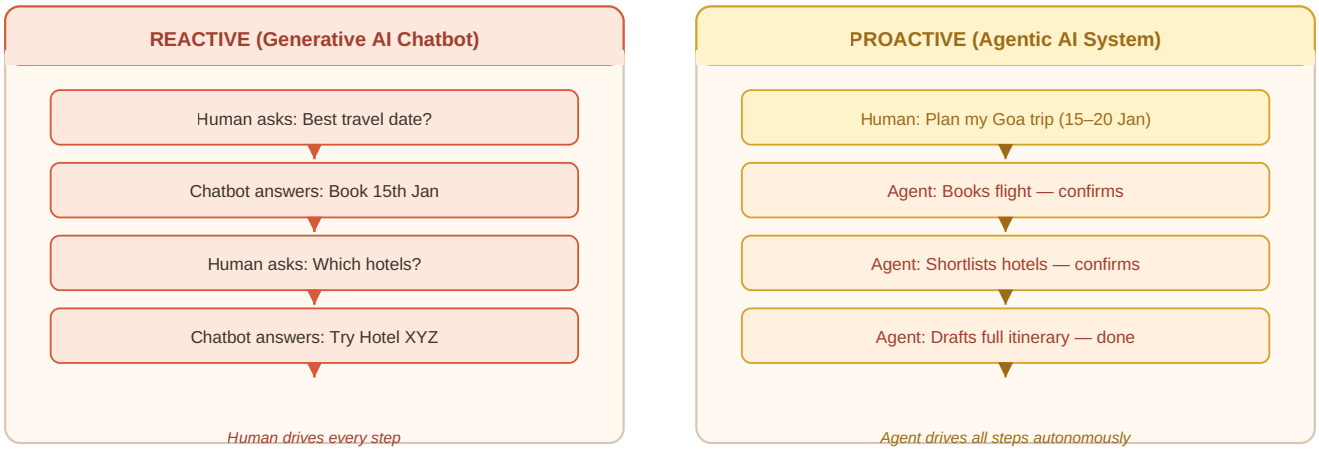

Figure 2.1 — Reactive GenAI requires a human prompt at each step; an Agentic AI system receives one goal and drives all subsequent steps independently.

IMPORTANT

Agentic AI does **not** replace Generative AI — it *orchestrates around it*. The LLM remains the reasoning engine (the "brain"), but the agentic layer adds planning, tool use, memory, and autonomous execution on top of that core LLM capability.

Practical Example: HR Recruiter Agent

Consider an HR recruiter who needs to hire a back-end engineer. With a conventional chatbot, the recruiter must ask a separate question for each step — drafting the JD, finding platforms, shortlisting candidates, scheduling interviews. With an Agentic AI system, the recruiter simply states the goal:

> *"I want to hire a back-end engineer with 2–4 years of experience, remote position."*

The agent then autonomously: drafts the job description, posts it on LinkedIn and Naukri, monitors applications, revises the JD when response is low, shortlists candidates via a résumé parser, schedules interviews via calendar API, sends offer letters, and initiates the onboarding workflow — checking in with the human only at high-stakes decision points (e.g., approving the offer letter content, confirming interview slots).

REAL WORLD INSIGHT

This level of autonomy is precisely what frameworks like **LangGraph** are designed to enable. Every step above — tool calls, memory retention, conditional routing, human approval checkpoints — maps directly to LangGraph primitives you will implement in later chapters.

↻ Quick Revision — What Is Agentic AI?

- Agentic AI receives a **goal** and autonomously plans and executes all steps to achieve it.
- Key difference from GenAI: Agentic AI is **proactive**; conventional chatbots are **reactive**.
- The human sets the goal once; the agent handles sub-tasks with minimal ongoing instruction.
- Agentic AI does not replace the LLM — it wraps planning, tools, and memory around it.
- Human-in-the-loop checkpoints are strategically inserted at high-risk decision points.

2. Six Key Characteristics of Agentic AI

Any AI application — chatbot, assistant, or agent — can be evaluated against six characteristics to determine whether it qualifies as a true Agentic AI system. If all six are present, the system is agentic.

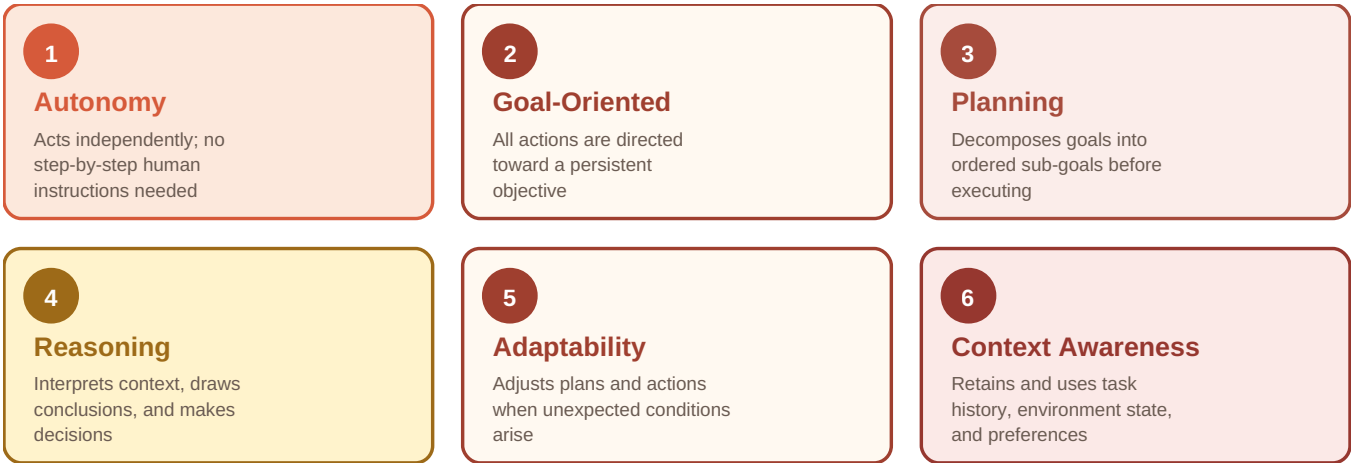

Figure 2.2 — The six defining characteristics of any Agentic AI system.

1 · Autonomy

Autonomy is the agent's ability to make decisions and take actions on its own to achieve a given goal — without needing step-by-step human instructions. It is the defining characteristic of Agentic AI and manifests across three dimensions:

- **Execution autonomy:** The agent carries out a multi-step plan without waiting for a prompt at each step.
- **Decision autonomy:** The agent decides how many candidates to shortlist, which résumés to reject, and what action to take next — without asking each time.
- **Tool-selection autonomy:** The agent independently chooses which tool to invoke (email API, calendar API, résumé parser) for each step.

COMMON MISTAKE

Giving an agent *unlimited* autonomy without guard rails is dangerous. An unconstrained agent might send offer letters with incorrect salaries, apply biased shortlisting criteria that violate employment law, or spend unbounded budget on promotional ads — all without asking permission. **Always define autonomy boundaries explicitly.**

CONTROLLING AUTONOMY

Four mechanisms exist to scope and constrain agent autonomy:

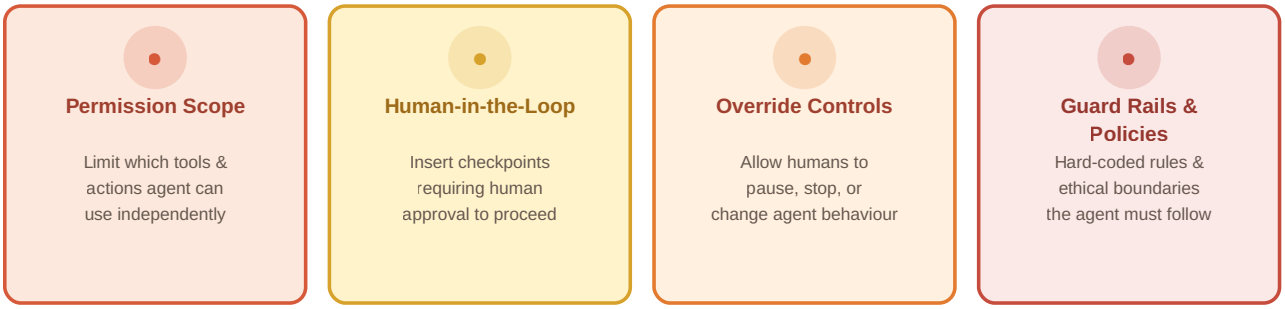

Figure 2.3 — Four mechanisms to constrain and control agent autonomy in production systems.

- **Permission Scope:** Define exactly which tools and actions the agent can use without asking. Example: the agent may screen résumés freely, but must ask before rejecting any candidate.
- **Human-in-the-Loop:** Insert approval checkpoints at high-risk steps. Example: the agent drafts the JD and posts it only after the recruiter confirms.
- **Override Controls:** Give the human the ability to pause, stop, or redirect the agent at any time. Example: issuing "pause hiring" immediately halts all agent activity.
- **Guard Rails & Policies:** Hard-code rules the agent must always follow. Example: "Never schedule interviews on weekends" or "Never use informal language in emails."

> **BEST PRACTICE**
>
> When building agentic systems with LangGraph, model all human approval checkpoints as explicit **interrupt nodes** in the graph, and define guard rails as system-prompt constraints on the LLM. This makes the autonomy boundary visible and auditable.

2 · Goal-Oriented Operation

A goal-oriented agent maintains a **persistent objective** throughout its entire operation. Every action it takes is directed toward achieving that objective — not just responding to an isolated prompt. The goal acts as a *compass* for autonomy: without a goal, the agent has no direction; without autonomy, the agent cannot pursue the goal independently.

Goals can be defined in two forms:

- **Unconstrained goals:** "Hire a back-end engineer." — open-ended objective.
- **Constrained goals:** "Hire a back-end engineer from India, remote-only, salary budget ₹30 LPA." — objective plus hard constraints the agent must respect.

Goals are stored in the agent's **core memory** as structured state — typically a JSON-like object containing the main goal, any constraints, current status (*active / completed / cancelled*), creation timestamp, and a progress tracker listing which sub-tasks have been completed.

Goals can also be **altered mid-execution**. If the recruiter decides to switch from a full-time hire to a freelancer after seven days with no applications, the agent discards the original plan, replans around the new goal, and begins executing the revised plan — without losing prior context.

INTERVIEW TIP

A strong answer to "What makes an AI system agentic?" will emphasize that agentic systems are *goal-driven*, not prompt-driven. The goal persists across the entire workflow and directs every sub-task and tool call. Mentioning that goals can carry *constraints* and can be *modified mid-execution* demonstrates depth.

↻ **Quick Revision — Autonomy & Goal-Oriented Operation**

- Autonomy means the agent acts, decides, and selects tools without per-step human prompts.
- Three levels of autonomy: **execution**, **decision**, and **tool-selection**.
- Autonomy is controlled via permission scope, human-in-the-loop, overrides, and guard rails.
- Goals give autonomy a *direction* — they are the compass of the entire agentic workflow.
- Goals are stored as persistent state and may include constraints; they can be changed mid-execution.

3. Planning and Reasoning

Planning — Breaking Goals Into Actions

Planning is the agent's ability to decompose a high-level goal into a structured sequence of sub-goals and actions. It is arguably the most important characteristic of Agentic AI, because all autonomous execution flows from a valid plan.

At a high level, every Agentic AI system operates in exactly two phases — and these phases are *iterative*, not one-shot:

1. **Planning Phase:** The agent receives a goal, reasons about it, and produces an ordered sequence of sub-goals and actions.
2. **Execution Phase:** The agent carries out that plan step by step, using tools and memory as needed.

If execution reveals that a step is impossible — a tool is unavailable, conditions have changed, or a step fails — the agent returns to the planning phase, replans, and tries again. This **plan → execute → replan loop** continues iteratively until the goal is achieved.

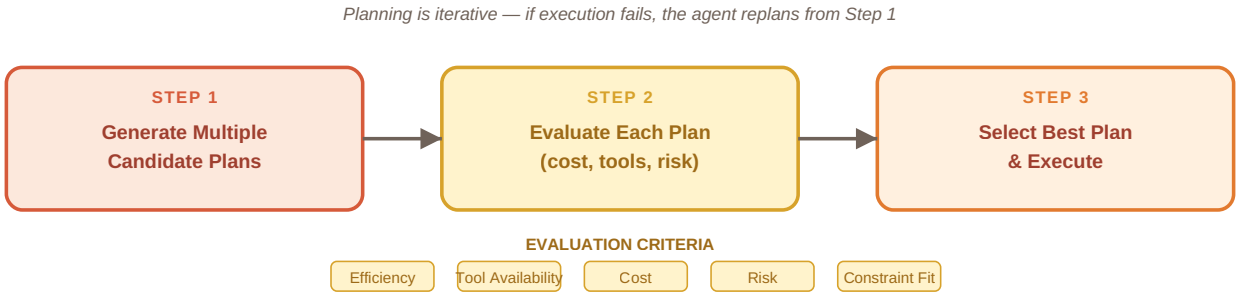

Figure 2.4 — The three-step planning process: generate candidate plans, evaluate against multiple criteria, then select and execute the best plan.

STEP 1 — GENERATE MULTIPLE CANDIDATE PLANS

Planning is a **search problem**: given an initial state (company needs an engineer) and a desired final state (engineer is hired), there are multiple paths between them. The agent generates several *candidate plans* rather than committing to a single approach immediately. For example:

- **Plan A:** Draft JD → post on LinkedIn, GitHub Jobs, and Indeed → promote with paid ads → screen applicants → interview → offer.
- **Plan B:** Run an internal referral programme → engage a recruitment agency → interview referred candidates → offer.

STEP 2 — EVALUATE EACH PLAN

The agent evaluates candidate plans across five dimensions:

Criterion	What the agent checks
Efficiency	Which plan reaches the goal fastest?
Tool Availability	Does the agent have all required tools? (e.g., if Google Search API is absent, any plan requiring it is eliminated.)
Cost	Which plan stays within budget constraints?
Risk	Which plan has the lowest probability of failure?
Constraint Alignment	Which plan best satisfies the goal's constraints (e.g., remote-only requirement)?

STEP 3 — SELECT THE BEST PLAN

Selection may happen via a **human-in-the-loop decision** (the agent presents the plans and the human picks one) or via a **pre-programmed policy** embedded in the agent's configuration. Once selected, the plan becomes the execution roadmap.

> **IMPORTANT**
>
> Planning is not a one-time event. If a step in the execution phase becomes impossible — for example, LinkedIn's API is down, or no candidates apply after two weeks — the agent *returns* to planning, generates new candidate plans for the remaining goal, and re-executes. The plan–execute loop is fundamentally **iterative**.

Reasoning — The Cognitive Engine

Reasoning is the cognitive process through which the agent interprets information, draws conclusions, and makes decisions — both during planning and during execution. It is what the LLM fundamentally provides.

Reasoning is required at *every stage* of the agent's operation:

Stage	Where reasoning is needed	Example
Planning	Goal decomposition	Breaking "hire an engineer" into JD → post → screen → interview → offer
	Tool selection per step	Deciding that "find salary benchmarks" requires a web search tool
	Resource & risk estimation	Estimating time, dependencies, and failure modes for each candidate plan
Execution	Decision-making between options	Should 2 or 3 candidates be shortlisted based on résumé quality?
	Human-in-the-loop judgment	Should the agent handle salary negotiation alone, or escalate to the recruiter?
	Error handling	LinkedIn is down — retry, notify human, or switch platform?

REAL WORLD INSIGHT

In LLM-based agentic systems, the reasoning capability comes from the underlying language model itself. Techniques like **Chain-of-Thought (CoT)** prompting and **ReAct** (Reason + Act) frameworks are specifically designed to make the LLM's step-by-step reasoning explicit, which dramatically improves the quality of both planning and execution decisions.

↻ Quick Revision — Planning & Reasoning

- Every Agentic AI operates in two iterative phases: **Plan** → **Execute** → (replan if needed).
- Planning is a search problem: the agent generates multiple candidate plans, evaluates them, and selects the best.
- Evaluation criteria: efficiency, tool availability, cost, risk, and constraint alignment.
- Reasoning powers *every* stage — goal decomposition, tool selection, decision-making, and error handling.
- If execution fails at any step, the agent returns to planning and replans around the new situation.

4. Adaptability and Context Awareness

Adaptability — Handling the Unexpected

Adaptability is the agent's ability to modify its plans, strategies, and actions in response to unexpected conditions — while remaining aligned with the original goal. It is what distinguishes a robust agentic system from a fragile scripted workflow.

Three categories of situations require adaptability:

- **Tool or Service Failures:** A required API (e.g., the calendar API) goes down mid-execution. The agent must detect the failure and find an alternative path — perhaps directly messaging the human to ask for their availability instead.
- **Adverse External Feedback:** The agent monitors its environment and detects that a metric has moved in the wrong direction (e.g., only two candidates applied after five days on LinkedIn). It adapts by revising the JD, broadening the role title, or launching paid promotions — proactively, without being asked.
- **Mid-execution Goal Changes:** The human changes the goal entirely — switching from hiring a full-time engineer to finding a freelancer. The agent discards the current plan, replans for the new objective, and executes the revised workflow.

> **BEST PRACTICE**
>
> Design agentic systems with explicit **feedback loops**: the agent should periodically observe the state of its environment and compare it to expected progress. In LangGraph, this is typically implemented as a monitoring node that can trigger a replan edge when KPIs fall below threshold.

Context Awareness — The Agent's Situational Memory

Context awareness is the agent's ability to understand, retain, and utilise relevant information across the entire duration of a multi-step, multi-day task. Without context awareness, an agent would "forget" its goal and prior actions every time a new interaction begins — making autonomous multi-step operation impossible.

A production agentic system must maintain the following types of context at all times:

Context Type	What it contains	Example
Original Goal	The primary goal and all constraints	"Hire a remote back-end engineer, 2–4 yrs exp"
Progress State	Which sub-tasks are complete, pending, or failed	JD posted ✓ · 8 applicants ✓ · 2 interviews scheduled ✓
Conversation History	Prior exchanges between agent and human supervisor	Recruiter approved JD on Day 1; requested wider net on Day 3
Environment State	Current real-world conditions the agent is monitoring	LinkedIn ad budget expires in 2 days; 8 total applicants
Tool Outputs	Results returned by tools during this session	"Candidate B: 3 yrs Django + AWS experience" from résumé parser
User Preferences	Established preferences and recurring decisions	HR prefers interview questions sent as Google Docs
Guard Rails	Active policies and ethical constraints	"Never send offers without approval"; "No weekend interviews"

MEMORY TYPES THAT IMPLEMENT CONTEXT AWARENESS

Context awareness is implemented through two types of *memory*:

- **Short-Term Memory:** Holds information relevant to the *current session* — active messages, current tool call outputs, and decisions made in this interaction. Analogous to human working memory. Example: "Candidate B has 3 years of Django experience" — relevant only while screening is active.
- **Long-Term Memory:** Persists *across sessions* — the original goal, user preferences, guard rails, completed task history, and strategic decisions. Example: "Never send offer letters without human approval" — a permanent policy that survives session restarts.

> **INTERVIEW TIP**
>
> When asked how agentic systems handle multi-day or multi-session workflows, the answer is **long-term memory**. The agent stores the goal, progress, and policies persistently. In LangGraph, this is implemented via **checkpointers** (e.g., SQLite or Redis persistence) that save the full graph state at each step, enabling the agent to resume exactly where it left off.

↻ Quick Revision — Adaptability & Context Awareness

- Adaptability covers three scenarios: **tool failures**, **adverse external feedback**, and **goal changes**.
- An adaptable agent proactively detects that things are going wrong and adjusts — without being prompted.
- Context awareness requires retaining: goal, progress, conversation history, environment state, tool outputs, preferences, and guard rails.
- **Short-term memory** = current session data; **Long-term memory** = cross-session persistence (goals, policies, history).
- In LangGraph, long-term memory is implemented with *checkpointers* that persist the full graph state.

5. Five Core Components of Agentic AI

Every Agentic AI application — regardless of framework or domain — is built from five fundamental components. Understanding these components is essential before building with LangGraph, because each LangGraph primitive maps directly to one or more of them.

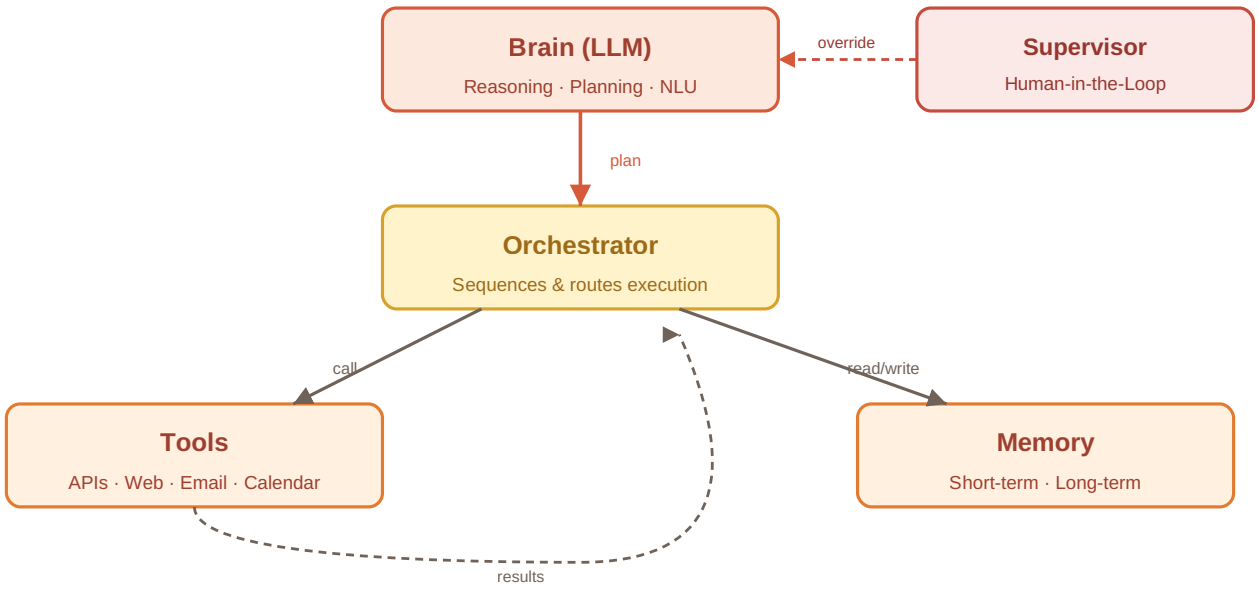

Figure 2.5 — The five core components of any Agentic AI system and how they interact. The Brain plans; the Orchestrator executes; Tools act on the world; Memory retains state; the Supervisor handles human oversight.

Component 1 — Brain (LLM)

The *Brain* is the LLM at the centre of the system. It is responsible for all cognitive heavy lifting:

- **Goal interpretation:** Understanding exactly what the user wants, including implicit constraints.
- **Planning:** Decomposing the goal into a structured sequence of sub-goals and actions.
- **Reasoning:** Making decisions at every step — which tool to call, what to do on failure, when to escalate.
- **Tool selection:** Deciding which tool to invoke and with what parameters for each step.
- **Natural language communication:** Generating and interpreting human-readable messages for the supervisor.

The Brain is the *only component that thinks*. All other components serve, extend, or manage the Brain's outputs.

> **IMPORTANT**
>
> The Brain (LLM) does **not** execute actions directly — it *decides* what actions to take. The Orchestrator is responsible for carrying out those decisions by invoking the appropriate Tools or routing to the appropriate next step.

Component 2 — Orchestrator

The *Orchestrator* is the execution manager — sometimes called the "project manager" of the agentic system. It takes the plan produced by the Brain and executes it step by step, handling all the runtime logic that surrounds execution:

- **Task sequencing:** Determines the order in which plan steps are executed.
- **Conditional routing:** Based on the output of one step, routes execution to one of several possible next steps.
- **Retry logic:** If a tool call fails (e.g., LinkedIn is down), retries with a configurable back-off strategy.
- **Looping and iteration:** Re-executes monitoring steps on a schedule (e.g., checking application counts every 24 hours).

- **Delegation:** Hands off tasks to the LLM (Brain) for reasoning, to Tools for external actions, or to the Supervisor for human approval.

In LangGraph, the Orchestrator is implemented as the compiled **StateGraph** — the graph itself defines what to execute and in what order, with conditional edges handling routing logic.

Component 3 — Tools

Tools are the agent's interface with the external world. Every action that changes something outside the agent — calling an API, reading a database, sending an email, performing a web search — is performed through a Tool. If the Brain is the mind, Tools are the hands and legs.

Tools used by our HR recruiter example:

- LinkedIn API — post and monitor job listings
- Naukri API — post job listings on the Indian platform
- Résumé parser — extract structured data from uploaded PDFs
- Calendar API — check recruiter availability, create interview slots
- Email API — draft and send emails to candidates and stakeholders
- Company HR Management System — retrieve salary bands, generate offer letters, initiate onboarding
- Knowledge Base (RAG) — retrieve relevant company policies, JD templates, and role requirements

> **REAL WORLD INSIGHT**
>
> A **RAG (Retrieval-Augmented Generation)** knowledge base is itself a special class of Tool — it allows the agent to retrieve factual, domain-specific information (salary ranges, policy documents, JD templates) at runtime rather than relying on the LLM's training data. In agentic systems, RAG grounding reduces hallucination for domain-specific decisions.

Component 4 — Memory

Memory is the storage layer that implements context awareness. Without memory, the agent forgets everything between interactions and cannot run multi-step, multi-day workflows. Memory has two operational tiers:

Memory Type	Scope	Typical Contents	LangGraph Equivalent
Short-Term	Current session only	Active messages, current tool outputs, immediate decisions	In-memory state (the current `State` dict)
Long-Term	Persists across sessions	Original goal, user preferences, completed task log, guard rails, strategic decisions	Checkpointer (SQLite, Redis, Postgres persistence)

Memory also handles **state tracking** — recording which sub-tasks have been completed, which are pending, and which have failed. This is what allows the agent to pick up exactly where it left off after a session ends or a failure occurs.

Component 5 — Supervisor

The *Supervisor* is the component that implements human oversight — it is how the human and the agent collaborate safely. The Supervisor handles three scenarios:

- **High-risk approval requests:** Before taking an irreversible or expensive action (sending an offer letter, launching a paid ad campaign), the agent pauses and requests explicit human approval via the Supervisor.
- **Guard rail enforcement:** The Supervisor intercepts any action that would violate a defined policy (e.g., scheduling a weekend interview) and blocks it before execution.
- **Edge case escalation:** When the agent encounters a situation outside its defined parameters — for example, a candidate with an exceptional résumé who doesn't meet the IIT/NIT requirement — the Supervisor flags the situation for human review rather than making an autonomous decision.

BEST PRACTICE

In LangGraph, the Supervisor pattern is built using **interrupt-before** or **interrupt-after** node annotations combined with a human-approval node. This creates an explicit, auditable approval workflow that can be inspected in the graph trace. Always implement the Supervisor as a first-class graph component rather than as an ad-hoc side effect.

↻ Quick Revision — Five Core Components

- **Brain (LLM):** Goal interpretation, planning, reasoning, tool selection, and NL communication.
- **Orchestrator:** Executes the plan — sequences tasks, handles routing, retries, loops, and delegation.
- **Tools:** The agent's interface to the external world — APIs, databases, email, search, RAG.
- **Memory:** Short-term (current session) + long-term (cross-session persistence via checkpointers).
- **Supervisor:** Implements human-in-the-loop — approval checkpoints, guard rail enforcement, edge-case escalation.

K. Key Takeaways

- **Agentic AI** is a paradigm in which an AI system receives a high-level goal and autonomously plans, executes, and adapts to achieve it with minimal human instruction.
- **Reactive vs. Proactive:** Conventional GenAI chatbots respond to each prompt in isolation; Agentic AI proactively drives a multi-step workflow from a single goal statement.
- **Autonomy** is the defining trait — but it must be explicitly controlled via permission scope, human-in-the-loop checkpoints, override controls, and guard rails.
- **Goal-Oriented:** The agent's entire operation is directed by a persistent goal, stored as structured state, which can include constraints and can be changed mid-execution.
- **Planning** is iterative: generate candidate plans → evaluate on efficiency/tools/cost/risk/constraints → select and execute → replan on failure.

- **Reasoning** (provided by the LLM) is required at every stage — goal decomposition, tool selection, decision-making during execution, and error handling.
- **Adaptability** handles tool failures, adverse environmental feedback, and mid-execution goal changes — always while staying aligned to the original goal.
- **Context Awareness** is implemented via short-term memory (session data) and long-term memory (persistent checkpointers), enabling multi-day autonomous workflows.
- **Five Components:** Brain (LLM) · Orchestrator · Tools · Memory · Supervisor — every LangGraph application maps its primitives to these five building blocks.

R. Revision Sheet

Agentic AI (Definition)

A software paradigm where the system receives one high-level goal and autonomously plans, executes, and adapts to achieve it — with human involvement only at designated checkpoints.

Planning

Decomposing a goal into ordered sub-goals. Three steps: (1) generate candidate plans, (2) evaluate on efficiency/tools/cost/risk/constraints, (3) select and execute. Iterative — replans on failure.

Reactive vs. Proactive

Reactive (GenAI): human prompts at every step, chatbot responds. Proactive (Agentic): human sets goal once, agent drives all subsequent steps.

Reasoning

The cognitive process (provided by the LLM) that enables goal decomposition, tool selection, decision-making, and error handling — at both planning and execution stages.

Autonomy

The ability to make decisions, select tools, and execute actions without per-step human guidance. Controlled via permission scope, human-in-the-loop, overrides, and guard rails.

Adaptability

Modifying plans and strategies in response to tool failures, adverse external feedback, or mid-execution goal changes — while staying aligned to the original objective.

Goal-Oriented

The agent maintains a persistent objective throughout its workflow. Goals are stored as structured state (with constraints, status, and progress) and may be altered mid-execution.

Context Awareness

Retaining and utilising goal, progress, conversation history, environment state, tool outputs, user preferences, and guard rails across a multi-step, multi-day workflow.

Short-Term vs. Long-Term Memory

Short-term: current session data (messages, tool outputs). Long-term: cross-session persistence (goal, preferences, completed task log, guard rails) via checkpointers.

Memory (Component)

Storage layer for context awareness. Short-term = in-memory State dict; Long-term = LangGraph checkpointer (SQLite, Redis, Postgres).

Brain (LLM)

The reasoning engine: interprets goals, creates plans, selects tools, makes decisions, and generates natural language. The only component that "thinks."

Supervisor

Implements human oversight — approval checkpoints for high-risk actions, guard rail enforcement, and escalation of edge cases to human review.

Orchestrator

The execution manager: sequences tasks, handles conditional routing, retries, loops, and delegation. Implemented in LangGraph as the compiled StateGraph.

Tools

The agent's interface with the external world — APIs, databases, email, calendar, web search, and RAG knowledge bases. Every external action goes through a Tool.

I. Interview Questions

Q1. How would you formally define Agentic AI, and how does it differ from a conventional Generative AI chatbot?

Q2. What are the six key characteristics that define an Agentic AI system? Give a one-sentence explanation for each.

Q3. An agent you deployed is sending offer letters with incorrect salary figures without asking for approval. What characteristic is misconfigured, and what mechanisms would you use to fix it?

Q4. Explain the planning process in an Agentic AI system. Why is it treated as a search problem, and what criteria are used to select the best plan?

Q5. Where exactly is reasoning required in an agentic workflow, and which component of the system provides that reasoning capability?

Q6. Describe three real scenarios that would trigger an adaptability response in an agentic system. How would the agent handle each one?

Q7. What is the difference between short-term and long-term memory in an agentic system? Give a concrete example of data that belongs in each type.

Q8. Name the five core components of an Agentic AI system and explain the role of each using a single practical example scenario.

Q9. What is the Supervisor component responsible for in an agentic system, and how is it typically implemented in LangGraph?

Q10. A candidate with an outstanding résumé does not meet a hard-coded requirement (must be from IIT/NIT). How should a well-designed agentic system handle this edge case, and which component manages it?

F. Further Reading

- **LangGraph Documentation — Concepts:** docs.langchain.com/langgraph/concepts — covers agents, memory, human-in-the-loop, and persistence patterns that directly implement the five components discussed in this chapter.
- **ReAct: Synergizing Reasoning and Acting in Language Models** (Yao et al., 2022) — the foundational paper behind the Reason + Act loop that powers LLM-based agents. Available on arXiv.
- **Anthropic Model Spec — Agentic Settings:** anthropic.com/model-spec — discusses autonomy, oversight, and safety considerations for agentic deployments from a leading AI lab perspective.
- **LangChain Blog — "What is an AI Agent?":** blog.langchain.dev — accessible overview connecting the theoretical characteristics covered here to LangChain/LangGraph implementation patterns.
- **Chain-of-Thought Prompting Elicits Reasoning in Large Language Models** (Wei et al., 2022) — the paper that demonstrated how explicit step-by-step reasoning in LLM prompts dramatically improves planning and decision quality in agentic systems.